Fortress AI

System & Dashboard Whitepaper

External technical review - Neural Ops UI v2 - Not investment advice

Fortress AI - System & Dashboard Whitepaper

Document purpose: External technical review of the Fortress AI platform, its operational architecture, and every dashboard panel with metric definitions, data provenance, and trading relevance.

Canonical filename: docs/FORTRESS_AI_WHITEPAPER.md
PDF export: docs/FORTRESS_AI_WHITEPAPER.pdf (generate via scripts/generate_whitepaper_pdf.py)
Version: Neural Ops UI v2 (June 2026)
Primary dashboard: Flask app at port 8050 (FORTRESS_AI_DASHBOARD_BUILD / FORTRESS_AI_DASHBOARD_PORT)
Disclaimer: Research and paper-trading infrastructure. Not investment advice.

1. Executive summary

Fortress AI is a multi-agent quantitative trading platform built around four distinct execution paths plus a Classic sibling stack for A/B comparison:

Layer	Agent / service	Role	LLM?
Intraday skim	Skim Swarm (fortress-ai-skim-swarm)	Always-on 1-minute rule-based long/short skims across 16 symbols	No
AI-infra overlay	Infra Swarm (fortress-ai-infra-swarm)	SRP stack propagation anchored on SMH; L1-L4 adaptive universe	No
Strategic AI	Unified AI Agent (fortress-ai-agent)	Periodic LLM-driven portfolio decisions on non-skim universe	Yes (DeepSeek)
Legacy SPY	SPY Intraday Agent	Ladder-based SPY session trading (optional; disabled when skim is primary)	No
Classic sibling	trading-bot repo	Rules-driven multi-agent stack on separate Alpaca account	Optional advisory

The **Command Center dashboard** is the single operational surface for monitoring all paths, diagnosing why trades did or did not occur, tracking P&L, governing self-improvement, and comparing Fortress AI against **Classic** (trading-bot).

Design principles (June 2026):

- Per-symbol independence - each skim/infra ticker owns parameters, pattern disables, and learning state.
- Deterministic hot-path adaptation - live recursive improvement without LLM latency in swarm loops.
- Tiered self-improvement - parameter tuning (Tier 0-1), prompt evolution (Tier 2), immutable risk rails (Tier 3).
- Recursive SI pipeline - integrity scan -> recommendation queue -> agent/human review -> optional autonomous code implementation.
- Full audit trail - every decision, block reason, cost, and governance action is JSONL-logged.

Related docs: docs/INFRA_SWARM.md, docs/UNIFIED_AI.md, docs/SELF_IMPROVEMENT.md, docs/COMPARISON_METRICS.md

2. System architecture

2.1 Data flow

External data (Alpaca, yfinance/IEX, SEC EDGAR, FRED, news sentiment, COT) feeds Skim Swarm, Infra Swarm, Unified AI, and Domain Ingest. Agents write to JSONL/JSON stores under data/. The Flask dashboard (dashboard/ai_command_center.py) aggregates state and serves the Neural Ops UI v2.

Classic Fortress metrics are read-only via utils/classic_bridge.py for comparison and SI capability review.

2.2 Production systemd services

Unit	Role
fortress-ai-dashboard	Neural Ops UI (:8050)
fortress-ai-skim-swarm	1m skim basket
fortress-ai-infra-swarm	AI-infra SRP overlay
fortress-ai-agent	Unified LLM agent
fortress-ai-rth-intraday-si	RTH autonomous SI loop (default 30 min)
fortress-ai-operator-status	15m operator snapshots
fortress-ai-governance	Daily governance maintenance (06:15 ET)

All schedules use America/New_York (FORTRESS_SYSTEM_TZ).

2.3 Refresh tiers

Tier	Interval	Mechanism	Panels affected
SSE	~5s	GET /api/stream/decisions	Header status, AI Mind, positions
Fast	60s + SSE	GET /api/ai/current_state	Market pulse, portfolio, beliefs
Medium	60s	Partial state patch	Domain intel, screener, LLM spend, market consciousness
Skim	45s	GET /api/skim/status	Skim swarm panel
Infra	45s	GET /api/infra/status	Infra swarm panel
Slow	300s	Ingest, SI, governance APIs	Ingest health, SI capability review, prompt evolution

Server-side cache: build_current_state() TTL ~25s.

3. Execution engines

3.1 Skim Swarm - core intraday engine

Universe (default 16 symbols): SPY, NVDA, MSFT, GOOG, AMZN, AAPL, SOXX, NASA, BRK.B, AGIX, AVGO, LLY, V, MA, PLTR, CRWD

Each symbol is an independent agent with:

- **4 patterns:** rip_fade (short), pullback_uptrend (long), momentum_long, momentum_short
- **Per-symbol params:** entry deltas, target multiplier, disable_patterns, side pauses, causation blocks
- **Dual-timescale learning:** 10-year daily historical seeds + live 1-minute session stats
- **Adaptive max open:** utils/adaptive_max_open.py scales effective max_open from market consciousness + session markers (base -> ceiling -> aggressive ceiling)

Recursive self-improvement loop (every 10 exits, then every 5, per symbol):

1. Disable PnL-negative patterns
2. Pause toxic sides (pause_long / pause_short)
3. Full symbol pause if session bleeds (pause_entries)
4. Tighten entry thresholds via causation keys (pattern|side|spy_regime|sym_regime|score_bucket)

3.2 Infra Swarm - AI stack SRP overlay

Purpose: Trade semiconductor / AI supply-chain propagation layers (SRP) anchored on SMH with adaptive L1-L4 universe.

Concept	Detail
Anchor	**FORTRESS_INFRA_ANCHOR (default SMH)**
Universe	**Adaptive rewrite via FORTRESS_INFRA_ADAPTIVE_UNIVERSE**
Layers	**L1-L4 stack mapping per symbol**
Session SI	**FORTRESS_INFRA_SWARM_SESSION_SI - pause entries on critical session**
Risk	**FORTRESS_INFRA_DAILY_STOP_USD, FORTRESS_INFRA_MAX_OPEN_POSITIONS, L1 gross cap**

Data: data/infra_swarm/decisions.jsonl, learned/, session_policy.json, adaptive_universe.json

3.3 Unified AI Agent - LLM strategic layer

Orchestrator: agents/unified_ai_agent.py
Execution package (June 2026 refactor):

Module	Role
unified_ai/position_manager.py	**Entry dedup, held-qty checks**
unified_ai/order_executor.py	**Chunked exits under notional cap**
unified_ai/risk_controller.py	**Oversized position detection**
unified_ai/legacy_flattener.py	**Periodic trim of legacy oversized lots**
unified_ai/settings.py	**Config from env + config/default.yaml**
utils/unified_enter_guard.py	**Enter cooldown + already_holding gate**
utils/order_chunking.py	**chunk_qtys, held_qty_for_symbol, notional cap**

Key env vars:

Variable	Default	Effect
FORTRESS_MAX_ORDER_NOTIONAL_USD	**50000**	**Max single-order notional; triggers chunked exits**
POSITION_DEDUPLICATION_ENABLED	**true**	**Block duplicate entries on same symbol**
FORTRESS_UNIFIED_ENTER_COOLDOWN_SEC	**(guard)**	**Cooldown between enter attempts**

Detectable log markers: already_holding, chunked_exit, enter_cooldown, entry_blocked_by_cooldown

Unified AI does **not** trade skim/infra universe symbols (FORTRESS_AI_SYMBOL_DENYLIST).

3.4 Market consciousness

Module: utils/market_consciousness.py
Purpose: Session intent, tape/VIX regime, alpha vs SPY, analogue days, proactive SI triggers. Feeds adaptive max-open and operator diagnostics.

Env: FORTRESS_MARKET_CONSCIOUSNESS, FORTRESS_CONSCIOUSNESS_CACHE_SEC
Data: data/market_consciousness/knowledge.json

3.5 RTH intraday autonomous SI

Script: scripts/rth_intraday_si_loop.sh -> scripts/rth_intraday_si.py -> utils/rth_autonomous_si.py

Each cycle (default 1800s, overridable by capability review):

1. Integrity scan (utils/integrity_diagnostics.py)
2. SI recommendation queue process
3. Edge scorecard / autofix
4. Swarm session SI
5. Per-symbol tune
6. Governance maintenance hooks

Env: FORTRESS_RTH_INTRADAY_SI, FORTRESS_RTH_SI_INTERVAL_SEC
Artifacts: data/rth_intraday_si/, data/integrity_scan_latest.json
Freeze: Operator halt sets SI-FROZEN: trading_halted - no auto-apply while halted.

3.6 Operator status monitor

Service: fortress-ai-operator-status
Script: scripts/operator_status_report.py
Interval: FORTRESS_OPERATOR_STATUS_INTERVAL_SEC (default 900 = 15 min)

Output: data/operator_status/latest.json, reports.jsonl

Snapshot fields: service health, skim/infra PnL & block mix, adaptive_max_open effective values, SI queue counts, auto-code health, top anomalies. Used for independent ops review (no dedicated dashboard panel).

3.7 Recursive SI recommendation queue

Pipeline: integrity scan -> utils/si_recommendation_queue.py -> optional utils/si_code_implementation.py (autonomous code SI)

Disposition	Meaning
auto_resolved	**Mitigation already deployed**
auto_applied	**Tier 0/1 auto-tune applied**
monitoring	**Watch only**
pending_agent_review	**Cursor agent triage**
pending_human_go	**Awaits explicit operator "go"**
auto_implement_queued	**Autonomous code runner queued**

Registry: config/si_fix_registry.json - finding codes, mitigation markers, effort/impact
Data: data/si_recommendation_queue.json, data/si_recommendation_summary.json

APIs:

- GET /api/si/recommendations
- POST /api/si/recommendations/process
- POST /api/si/recommendations/<id>/agent-assess|human-go|implemented
- POST /api/si/code/run, POST /api/si/code/implement/<id>

Autonomous code env: FORTRESS_SI_AUTO_CODE, FORTRESS_SI_AUTO_CODE_MAX_PER_DAY, FORTRESS_SI_AUTO_COMMIT, FORTRESS_SI_AUTO_PUSH, FORTRESS_SI_AUTO_CODE_REQUIRE_E2E, CURSOR_API_KEY

3.8 SI capability review + Classic bridge

Module: utils/si_capability_review.py - cross-stack objectives (Fortress AI + Classic)
Bridge: utils/classic_bridge.py - read-only Classic PnL, fill recency, daily screen health, open positions
Overrides: data/si_capability/overrides.json
APIs: GET /api/si/capability-review, POST /api/si/capability-review/run, GET /api/si/singularity

Classic env: CLASSIC_DATA_DIR, FORTRESS_TRADING_BOT_ROOT, CLASSIC_PNL_LEDGER_PATH

4. Dashboard panels - detailed reference

Template root: dashboard/templates/ai_dashboard.html
JS: dashboard/static/js/ai-dashboard.js

Panel 0: Build stamp banner

Purpose: Deployment verification - confirms the browser loaded the expected UI bundle.

Metric	Source	Why it matters
ui_build	Env FORTRESS_AI_DASHBOARD_BUILD	Prevents reviewing stale UI after deploy

Panel 1: Header bar

Purpose: Operational command center - instance identity, neural state, portfolio headline, operator controls.

Metric	Field	Source	Trading importance
Instance name	state.instance	FORTRESS_INSTANCE_NAME	Multi-VM identification
Mode badge	state.dry_run	FORTRESS_AI_DRY_RUN	DRY-RUN = no live orders
Skim universe count	skim_preview.universe.length	FORTRESS_SKIM_UNIVERSE	Active basket size
Neural state	state.ui_status	_infer_ui_status()	WAITING -> OBSERVING -> THINKING -> EXECUTING
Portfolio equity	state.portfolio.equity	Alpaca get_account()	Primary capital metric
API spend today	state.today_llm_spend_usd	ai_llm_cost_ledger.jsonl	LLM cost control
Halt button	POST /api/operator/halt	operator_trading_halt.json	Emergency kill switch
Run AI cycle	POST /api/agent/run-cycle	On-demand unified agent	Manual trigger

Panel 2: Skim Swarm (live)

API: GET /api/skim/status (45s poll)
Template: components/skim_swarm_panel.html
Purpose: Real-time view of the always-on intraday skim basket - primary production path.

Summary tiles

Metric	Source	Trading importance
Daily realized / unrealized / session net	decisions.jsonl + Alpaca	Today's P&L
Exit count	decisions.jsonl	Adaptation sample size
Open positions	Alpaca + state	vs adaptive max_open effective
HALTED	swarm_state.json	Daily stop gate

Per-symbol table

Side, price/change, unrealized/realized, W/L/exits, last action - from learned/*.json and Alpaca.

Panel 3: Infra Swarm (AI stack SRP)

API: GET /api/infra/status (45s poll)
Template: components/infra_swarm_panel.html
Purpose: Monitor AI supply-chain overlay - stack stress, layer assignments, per-symbol P&L.

Metric	Source	Trading importance
Anchor / universe	infra_swarm_config	Active SRP basket
Daily realized	infra decisions + pnl	Infra session P&L
Open positions	latest_metric	vs infra max_open effective
Stack signal / stack_stress	stack_signal	Propagation stress indicator
Per-symbol layer	layer_for_symbol	L1-L4 stack role
dry_run	infra config	Safety mode

Panel 4: Why no trade?

API: trading_diagnostics in current_state; GET /api/trading/diagnostics
Template: components/why_no_trade.html
Purpose: Diagnostic - why blocked/skipped trades occurred (14-day lookback).

Skim column: waves, entry proposed/executed/rate, HALTED, day P&L, **block_reason_counts** (pattern_disabled, pause_entries, spread_too_wide, causation_blocked, cooldown, no_edge, max_open_positions, eod_force_flatten).

Fortress AI column: dry_run, min_confidence, executed/cycles, block_reason_counts from ai_decisions.jsonl (includes already_holding, entry_blocked_by_cooldown).

Computation: utils/trading_diagnostics.py

Panel 5: Market consciousness

API: GET /api/ai/market_consciousness
Template: components/market_consciousness.html
Purpose: Session-level market read driving adaptive participation and SI triggers.

Metric	Trading importance
session_intent.plan_line	Operator narrative for today's posture
participation_target	How aggressively to trade
consciousness_posture.mode	Risk-on / neutral / defensive
temporal.slot_key	RTH phase (open, mid, close)
self_state.alpha_vs_spy_pct	Fortress vs benchmark
market_tape / vix_regime	Regime context
session_diary entries/exits	Activity vs intent
analogue_days	Historical similarity
market_events	Scheduled catalysts
proactive_si_trigger	Auto-SI escalation flag

Panel 6: AI Mind (column 1)

Template: components/ai_mind.html
Purpose: Unified LLM Agent reasoning, confidence, beliefs.

Market assessment, chain of thought, confidence vs min_confidence gate, trade belief memory (P7), recent decisions - from ai_decisions.jsonl and beliefs/beliefs.json.

Panel 7: Market pulse / Active scan / Positions (column 2)

Template: components/market_view.html

Section	Key metrics	Source
Market pulse	SPY price/change, VIX, RSI(14)	yfinance, charts API
Active scan	Symbol chips, screener source	AI screen_market or Classic fallback
Positions	Sym, Qty, Entry, Last, P&L	Alpaca get_all_positions()

Panel 8: Performance / API usage / System / Domain intel / Ingest (column 3)

Template: components/intelligence.html

LLM spend chart, API cap bar, system checklist (kill switch, Alpaca, SSE), domain intel regime hints, ingest health (SEC/FRED/News/COT).

Panel 9: Self-improvement (Tier 0-1)

APIs: /api/self_improvement/
*Template:** components/self_improvement.html

Parameter bounds: confidence_threshold, decision_interval, rsi_entry_threshold. Flow: Propose -> Shadow -> Approve/Reject -> Monitor -> Auto-revert.

Panel 10: SI capability review

APIs: GET /api/si/capability-review, POST /api/si/capability-review/run
Template: components/capability_review.html
Purpose: Cross-stack outcome review vs objectives (Fortress AI + Classic).

Metric	Trading importance
objective_gaps	Unmet SI targets by component
intervention_success_rate	SI effectiveness score
effective_rth_interval_sec	Adaptive RTH SI cadence
stack_metrics	Skim/infra/classic rollup
classic_recommendations	Suggested Classic interventions

Panel 11: Prompt evolution (Tier 2)

APIs: /api/prompt_evolution/
*Template:** components/prompt_evolution.html
Additive appendix only - cannot modify schema or risk params.

Panel 12: Governance (Tiers 0-3)

APIs: /api/governance/
*Template:** components/governance_panel.html

Tier	Risk	Behavior
0	Low	Auto-approve if shadow criteria met
1	Medium	24h human veto window

Tier	Risk	Behavior
2	Prompt	Human approval via prompt evolution
3	Immutable	Blocked - position size, stops, exposure, pre_trade_gate

Panel 13: Classic vs AI comparison drawer

API: GET /api/comparison
Template: components/comparison_drawer.html
Side-by-side Fortress AI vs Classic on separate Alpaca accounts: equity, P&L, win rate, trades/cycles, opportunity flags, symbol overlap, Classic fill-recency metrics.

Panel 14: Expert mode (overlay)

Trigger: Header Expert or key E | **API:** GET /api/expert/bundle
Tabs: Prompt note, last decision JSON, cost ledger tail, system logs, raw state JSON.

Panel 15: SPY Intraday (not mounted in live UI)

Template and APIs exist; typically disabled when skim swarm is primary.

5. Data architecture

Path	Written by	Read by
data/skim_swarm/decisions.jsonl	Skim agent	Skim panel, diagnostics
data/skim_swarm/edge_scorecard.json	Edge scorecard	RTH SI, dashboard
data/skim_swarm/session_policy.json	Swarm session SI	Entry pause / caps
data/infra_swarm/decisions.jsonl	Infra agent	Infra panel
data/infra_swarm/session_policy.json	Infra session SI	Infra caps
data/ai_decisions.jsonl	Unified AI	AI Mind
data/ai_llm_cost_ledger.jsonl	API costs	API usage
data/beliefs/beliefs.json	Belief manager	Trade belief memory
data/operator_trading_halt.json	Operator	Kill switch
data/operator_status/latest.json	Operator status cron	External ops review
data/si_recommendation_queue.json	SI queue	SI APIs, operator status
data/si_recommendation_summary.json	Queue rollup	Agent triage
data/si_capability/overrides.json	Capability review	Meta-knobs
data/unified_ai/enter_guard.json	Enter guard	Cooldown state
data/market_consciousness/knowledge.json	Market consciousness	Dashboard panel
data/integrity_scan_latest.json	Integrity scan	RTH SI
config/si_fix_registry.json	Fix registry	Deployment detection

6. Security and risk controls

Control	Mechanism
Kill switch	operator_trading_halt.json
Daily stop	FORTRESS_SKIM_DAILY_STOP_USD / FORTRESS_INFRA_DAILY_STOP_USD
Session SI critical	pause_new_entries - no new swarm entries; exits continue
Adaptive max open	Scales skim/infra caps from consciousness + session markers
Orphan universe guard	Entries blocked outside FORTRESS_*_UNIVERSE
Edge gates	RR / cost / expectancy (FORTRESS_EDGE_*)
Position deduplication	already_holding + enter cooldown (Unified AI)
Chunked exits	FORTRESS_MAX_ORDER_NOTIONAL_USD splits oversized orders
Legacy flatten	RiskController every 5 min on Unified AI boot path
Bracket tick clamp	Alpaca min $0.01 offset on bracket stops
Spread filter	FORTRESS_SKIM_MAX_SPREAD_BPS (25)
EOD flatten	Force flat before close
Weekly LLM cap	FORTRESS_AI_WEEKLY_COST_CAP_USD
Symbol denylist	Unified blocked from skim/infra universe
Confidence floor lock	FORTRESS_AI_CONFIDENCE_FLOOR_LOCK
SI freeze on halt	RTH SI skips auto-apply when operator halt active
Infra L1 gross cap	Stack-level long exposure limit

Protected (immutable): utils/pre_trade_gate.py, utils/operator_halt.py - SI cannot weaken.

7. Known limitations

1. Skim and Classic share one Alpaca account - skim max_open_positions can saturate when Classic holds positions.
2. No billing module - LLM cost tracking only.
3. SPY panel unwired when skim is primary.
4. Historical verify uses daily bars; live skim uses 1-minute bars.
5. Paper trading on Alpaca paper accounts.
6. Infra /api/infra/status may return partial data when service is idle.

8. External review checklist

Reviewers should evaluate:

- **Architecture:** Is four-path separation (skim / infra / unified / classic) clean? Are agent responsibilities modular?
- **Risk:** Are dedup, chunked exits, and adaptive max-open coherent under shared-account constraints?
- **SI governance:** Is the queue workflow (agent review -> human go -> implement) sufficient before code changes ship?
- **Execution:** Are block_reason markers sufficient for postmortems?
- **Reliability:** Do operator status + RTH SI cycles provide enough visibility without dashboard overload?
- **Comparison:** Is Classic bridge data sufficient for fair A/B review?

9. Glossary

Term	Definition
Pattern	rip_fade, pullback_uptrend, momentum_long, momentum_short
Winning pattern	Positive net PnL after min samples - not trade win rate
SRP / Infra	Semiconductor supply-chain propagation overlay

Term	Definition
Causation key	Context bucket blocking repeated losing entries
Belief (P7)	Persistent learned fact from closed trades
Neural state	WAITING / OBSERVING / THINKING / EXECUTING
SI queue	Recursive self-improvement recommendation workflow
chunked_exit	Exit split into child orders under notional cap

Fortress AI - Research dashboard - Not investment advice